An Educational Framework for AI-Driven RF Signal Processing on FPGA

Jude Eschete
Dept. of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, NJ, USA
JEschete@Stevens.edu

Project Advisor: Bernard Yett
Dept. of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, NJ, USA
BYett1@Stevens.edu

***Abstract*—University electrical engineering programs typically deliver instruction in FPGA digital design, RF communications, embedded systems, and machine learning as independent courses, leaving graduates without structured experience integrating these disciplines. This paper presents an educational framework built around a benchtop hardware platform in which STM32 microcontrollers paired with RFM95W LoRa transceivers transmit RF signals that are received and processed in real time by a Digilent Nexys A7-100T FPGA, with collected signal data feeding a machine learning classification pipeline. A five-module progressive curriculum sequences students from hardware bring-up and embedded firmware development through FPGA RTL design, system integration, and AI-driven signal analysis. Each module makes the interfaces between disciplines explicit teaching objectives, enabling students to reason about how design decisions in one domain propagate constraints into adjacent ones.**

***Index Terms*—FPGA, LoRa, machine learning, RF signal processing, embedded systems, curriculum design, signal classification**

I. Introduction

Modern electrical and computer engineering practice increasingly demands fluency across multiple technical domains simultaneously. Defense, telecommunications, and aerospace systems routinely integrate real-time RF data acquisition, hardware-accelerated digital processing, embedded firmware, and intelligent signal analysis within a single pipeline. University curricula, however, frequently deliver these disciplines in isolation. A student may learn HDL synthesis and timing closure in a digital design course, modulation theory and channel models in a communications course, interrupt-driven firmware in an embedded systems course, and classification on benchmark datasets in a machine learning course—without encountering a structured experience that requires synthesizing these skills into a working system.

This isolation creates a gap between academic preparation and professional practice. Engineers who understand each domain individually often struggle with the interface problems that arise when combining them: how RF modulation parameters affect FPGA buffer sizing, how reconfigurable logic resource constraints shape a data collection pipeline for machine learning, or how real-world RF propagation deviates from classroom channel models. These cross-disciplinary concerns are not incidental details but core engineering competencies that no single-domain course addresses.

Prior research has contributed to individual components of this problem but has not combined them into an integrated educational experience. On the pedagogical side, cloud-based FPGA toolchains with structured lab sequences have lowered the barrier to digital design education [9], and the STM32 Nucleo platform has been validated as an effective embedded systems teaching tool with significantly greater peripheral capability than Arduino-class boards [10]. On the technical side, FPGA-based RF signal processing and inference have advanced considerably: Boolean reservoir computing achieves CNN-competitive modulation classification accuracy with far fewer trainable parameters [1], streaming CNN architectures on RFSoC FPGAs classify eight modulation schemes continuously at 128 MHz [3], FPGA-accelerated ResNet reduced angle-of-arrival inference cycle count by 28.2% over CPU while improving robustness to multipath fading [2], and SoC FPGA LoRa gateways have demonstrated hardware-accelerated processing within tight resource budgets [7]. For LoRa signal classification specifically, deep learning has proven effective across multiple tasks: CNNs on spectrogram inputs achieved 99.77% spreading factor detection across a -20 dB to +30 dB SNR range [5], CNNs on raw I/Q data reached 93.3% device identification accuracy under real experimental conditions [6], neural network demodulation matched coherent matched-filter performance while tolerating carrier frequency offset [8], and comparative studies of DNN, CNN, and DBN architectures provide empirical guidance on accuracy-complexity tradeoffs for resource-constrained deployment [4]. Taken together, this body of work establishes the technical feasibility of every subsystem in the proposed platform. What is absent is a curriculum that sequences these subsystems into a unified learning experience where the connections between them are the primary subject of instruction.

The contributions of this paper are: (1) a five-module progressive educational curriculum that integrates FPGA digital design, RF communications, embedded firmware, and machine learning into a single hardware-grounded learning sequence; (2) a concrete benchtop reference platform built from commercially available components that instantiates each cross-domain interface as an experimentally observable design problem; and (3) a custom software engagement simulator that provides students with a physics-grounded, labeled training

environment before physical hardware is available. Section II formally defines the research problem and its context. Section III presents the framework in detail.

II. Problem Statement

The research problem is the absence of a structured, progressive educational curriculum that integrates FPGA digital design, RF communications, embedded microcontroller programming, and machine learning-based signal analysis into a unified learning experience where each cross-domain interface is a deliberate teaching objective.

The problem has a strict sequencing constraint that single-domain curricula do not impose. A student cannot meaningfully study the RF-to-FPGA interface without first having a working beacon, cannot study the FPGA-to-data-pipeline interface without first having a working RF-to-FPGA interface, and cannot train a useful classifier without first having a labeled dataset collected from that pipeline. Hardware bring-up must therefore come first, and every downstream learning objective inherits the correctness of what preceded it. This dependency chain is the reason the framework is organized as a progressive module sequence rather than a set of parallel tracks.

The target population is senior undergraduate and master's-level electrical engineering students who have completed introductory coursework in each of the four domains individually. These students possess the foundational prerequisites—HDL syntax, basic modulation and link theory, embedded C programming, and introductory machine learning—but lack experience working with systems that require all four simultaneously. When confronted with a multidisciplinary design problem, they face a secondary difficulty: understanding how a decision made within one domain generates requirements and constraints in adjacent domains that cannot be resolved from within a single discipline.

The concrete system context is a benchtop RF signal processing testbed. Multiple STM32 Nucleo-L476RG microcontrollers, each paired with an RFM95W LoRa transceiver, serve as independently configurable beacon transmitters. A receiver station captures incoming transmissions and forwards parsed RF data to a Digilent Nexys A7-100T FPGA, which performs real-time detection, packet integrity verification, and feature extraction in programmable logic on the Artix-7 fabric. Extracted signal features are collected by a host machine and used to train and evaluate machine learning classifiers for signal classification and emitter identification tasks. This architecture exposes four concrete interface problems that span disciplinary boundaries: the firmware-to-RF interface, the RF-to-FPGA interface, the FPGA-to-data-pipeline interface, and the data-to-model interface. Each requires cross-disciplinary reasoning to characterize and resolve.

The specific problem being solved is not a shortage of technical resources in any individual domain. Comprehensive textbooks, open-source FPGA IP cores, well-documented ML frameworks, and hardware reference designs exist for every component in this system. The gap is the absence of a curriculum that sequences these resources into an integrated learning experience, identifies the interface problems between domains as first-class subjects of instruction, and provides students with a concrete hardware implementation through which to develop and test their understanding.

The problem extends to the educator as well. An instructor preparing a multidisciplinary course of this kind cannot simply draw on deep expertise in a single domain—they must understand how each discipline's design decisions affect the others well enough to write labs, anticipate failure modes, and evaluate student work at the integration points. In practice, few faculty have equivalent depth across FPGA digital design, RF communications, embedded firmware, and machine learning simultaneously; most come to one or two of these domains as relative outsiders. Without a pre-existing framework that makes the cross-domain interfaces explicit—specifying what the integration problems are, why they arise, and how they can be observed on real hardware—the course preparation burden alone may be enough to prevent such a curriculum from being developed at all. The framework proposed in this paper addresses both sides of this problem: it provides students with the integrated learning experience they currently lack, and it provides educators with the structured reference they would need to deliver it.

III. Proposed Integrated Educational Framework

The proposed solution is a five-module curriculum paired with the benchtop hardware platform described in Section II. The framework assumes foundational domain knowledge and targets the integration layer: the design decisions, interface protocols, and system-level reasoning skills that emerge only when students work across disciplinary boundaries simultaneously.

A. Hardware Platform

The platform was designed around three subsystem layers, with each hardware selection driven by what it asks students to think about rather than by peak performance.

The transmission layer consists of STM32 Nucleo-L476RG microcontrollers, each paired with an RFM95W LoRa transceiver operating in the 915 MHz ISM band. LoRa was chosen as the RF modality because its chirp spread spectrum waveform exposes a small set of directly tunable parameters—spreading factor, bandwidth, and coding rate—whose effects on link behavior are large enough to observe with benchtop equipment. A student who changes the spreading factor from SF7 to SF12 gains roughly 7.5 dB of link margin per step but cuts the data rate in half [12]; that single decision ripples downstream into FPGA buffer sizing, dataset collection time, and ultimately the variance of the ML training set. This chain of consequences, spanning all four disciplines, is exactly the kind of reasoning the framework is designed to develop. The STM32 platform supports this goal by providing enough peripheral flexibility—multiple independent SPI, I2C, and USART interfaces with DMA support—that students can make and measure the impact of real firmware design decisions

TABLE I
RF TRANSCEIVER TRADE STUDY SUMMARY.

Module	Chipset	Interface	Outcome
Adafruit RFM95W	SX1276	SPI, 3.3 V	**Selected**
AI-Thinker Ra-02	SX1278	SPI, 433 MHz	Rejected: band
EByte E32-915T20D	SX1276	UART only	Rejected: abstraction
HopeRF RFM96W	SX1276	SMD, no brk.	Rejected: form factor

rather than working around hardware limitations [10], [13]. Prior work confirms that the RF signatures produced by LoRa hardware are rich enough to support device identification [6] and neural network demodulation [8] at the analysis layer, validating LoRa as a suitable signal source for the full pipeline.

At the processing layer, the Digilent Nexys A7-100T serves as the FPGA development platform. Its Artix-7 fabric—240 DSP slices, approximately 4,860 Kb of block RAM, and 63,400 LUTs [11]—is large enough to implement a complete signal processing datapath but constrained enough that students must make deliberate resource allocation decisions. This is an important design choice: a platform with unlimited resources would hide the tradeoffs that make FPGA design interesting. Students learn to partition logic between the serial ingestion front-end, the packet parsing and CRC pipeline, and the feature extraction back-end within a fixed budget, which is the same kind of reasoning required in professional FPGA development. The board connects to the STM32 receiver through its PMOD headers, giving students a concrete physical interface to characterize and debug. Prior work has shown that inference pipelines of comparable complexity fit well within the resource envelope of similar devices [1], [3], confirming that the Artix-7 is not a limiting factor for the scope of this curriculum.

The RFM95W was selected after a weighted trade study against three alternative sub-GHz transceiver modules, summarized in Table I. Selection criteria were weighted toward interface compatibility (20%), frequency band (15%), modulation flexibility (15%), documentation quality (15%), cost (10%), power (10%), on-chip packet engine features (10%), and form factor (5%). The RFM95W won on every dimension that matters pedagogically: it exposes a 3.3 V SPI interface the Nucleo can drive directly, operates in the 915 MHz U.S. ISM band, supports both LoRa and FSK/OOK for multi-modulation classification exercises in later modules, and ships as a breadboard-ready breakout with extensive documentation. The EByte E32-915T20D was rejected because its UART-only interface hides the register-level radio programming that Module 1 is designed to teach; the AI-Thinker Ra-02 was rejected for operating at 433 MHz outside the U.S. ISM band; and the HopeRF RFM96W was rejected for requiring a custom breakout.

At the analysis layer, a host machine accumulates the labeled feature vectors exported by the FPGA and serves as the ML development environment. Students select and train classifiers against this hardware-collected dataset, guided by prior comparative results on RF modulation classification [4] and LoRa-specific deep learning benchmarks [5] as reference points for evaluating their own models.

B. Engagement Simulator

To bridge the gap between curriculum introduction and physical hardware bring-up, a custom software engagement simulator was developed in Python as a standalone pre-hardware learning tool. The simulator models a multi-asset electronic warfare (EW) scenario—two cooperative target stations and an adversarial jamming threat—using a tick-based event engine that processes communication, jamming, and counter-jamming interactions at a configurable rate (500 ms per tick by default).

The simulator incorporates a physics-grounded RF model directly connected to the theoretical content students encounter in Module 1. Free-space path loss (FSPL) is computed per the standard Friis equation, and maximum communication range is derived from a link budget using configurable transmit power, antenna gain, and receiver sensitivity parameters. Target assets are modeled as frequency-hopping spread spectrum (FHSS) radios operating at 225 MHz (UHF military band) with eight available channels and a seed-synchronized hop sequence that both target stations share; the jamming threat uses a wideband receiver and DRFM-style noise jammer. This FHSS model connects directly to the spectral and synchronization concepts students work with during FPGA RTL design in Module 2, where clock domain management and frequency-selective filtering are primary design challenges.

Asset positions are specified in WGS-84 geodetic coordinates (latitude, longitude, altitude), with haversine-based range computation and geographic visualization on a 2-D map canvas. The default scenario places assets at coordinates representative of a realistic operational area, giving students immediate exposure to the link budget reasoning—how transmitted power, range, and receiver sensitivity interact—that governs the LoRa beacon configurations they will set in Module 1.

At each simulation tick, the engine resolves target-to-target communication attempts, threat jamming actions, and target counter-jamming responses in sequence. The outcome of each event is determined by the current channel alignment, effective range, and stochastic counter-jamming probability (default: 40%). All events are written to a dual-log output: `engagement.log` records timestamped event descriptions, and `data_stream.log` records the raw binary sequences exchanged—structured DATA, ACK, JAM, and counter-JAM patterns. These labeled logs serve as the first training dataset available to students in Module 4, allowing ML pipeline development to begin before any hardware RF data has been collected. Critically, the ground-truth labels (communication success, jamming state, counter-jamming outcome) are embedded directly in the log structure, giving students a clean supervised learning problem on which to validate preprocessing and model training code before the noisier, real-world dataset from the FPGA is available.

TABLE II
BEACON PACKET FORMAT (32 BYTES).

Field	Offset	Size
Preamble sync (`0xAE5A`)	0	2 B
Beacon ID	2	1 B
Sequence number	3	2 B
TX power (dBm)	5	1 B
Modulation code	6	1 B
Spreading factor	7	1 B
Bandwidth code	8	1 B
Timestamp (ms)	9	4 B
Payload	13	17 B
CRC-16/CCITT-FALSE	30	2 B

TABLE III
RECEIVER-TO-FPGA FORWARDING FRAME (43 BYTES).

Field	Offset	Size
Start delimiter (`0x7E`)	0	1 B
Frame length	1	1 B
RX timestamp (ms)	2	4 B
RSSI (dBm, two's comp.)	6	1 B
SNR (0.25 dB, signed)	7	1 B
Frequency error (Hz, int16)	8	2 B
Beacon payload (Table II)	10	32 B
Frame CRC-8	42	1 B

TABLE IV
FEATURE VECTOR FORMAT (32 BYTES, ONE BRAM ROW).

Field	Offset	Size
Beacon ID	0	1 B
Modulation code	1	1 B
Spreading factor	2	1 B
Reserved	3	1 B
RSSI (latest)	4	2 B
SNR (latest)	6	2 B
Frame count	8	4 B
Last arrival timestamp	12	4 B
Inter-arrival time	16	4 B
RSSI delta	20	4 B
Priority score	24	4 B
Reserved / padding	28	4 B

Per-tick performance metrics—communication success rate, jamming effectiveness, and counter-jamming rate—are accumulated over a rolling 60-tick history and displayed as live charts. These same three metrics form the primary benchmarks students measure on the physical system in Module 5, so the simulator establishes the measurement methodology and expected performance envelope before students have seen any hardware results.

C. Cross-Domain Frame and Feature Formats

The four cross-domain interfaces identified in Section II are made concrete by three fixed-length data structures that every student implements: a 32-byte beacon packet emitted by the STM32 transmitter firmware, a 43-byte forwarding frame wrapped around it by the STM32 receiver before handoff to the FPGA, and a 32-byte feature vector produced by the FPGA and consumed by the downstream machine learning classifier. These artifacts are the primary instructional objects of the framework—every field in each one corresponds to a design decision made in one disciplinary domain and observed in another.

Table II shows the beacon packet. The metadata fields (beacon ID, TX power, modulation code, spreading factor, bandwidth code) serve a dual role: they give the downstream ML pipeline supervised ground-truth labels, and they let the FPGA verify its own detection logic against values the beacon claims it transmitted. The 17-byte payload region carries either a pseudo-random sequence for bit-error-rate characterization or a structured pattern that exercises specific demodulator behavior, and the trailing CRC-16/CCITT-FALSE enables the receiver to distinguish link errors from downstream corruption.

Table III shows the receiver-to-FPGA forwarding frame. The receiver prepends a `0x7E` start delimiter, a length byte, an RX timestamp, and three SX1276-derived measurement fields (RSSI, SNR, frequency error) that are not available to the beacon itself. The full 32-byte beacon packet is carried verbatim as the payload, and a frame-level CRC-8 protects the transport layer between the STM32 receiver and the FPGA independently of the inner LoRa CRC. At 115200 8N1 the 43-byte frame plus framing overhead yields a steady-state capacity of approximately 28 frames per second, which comfortably supports the two-beacon default configuration and scales by a factor of three before SPI handoff becomes necessary in Module 2.

The third cross-domain artifact is the feature vector produced by the FPGA ingestion pipeline and consumed by the downstream machine learning classifier. Table IV shows its layout: a 32-byte row aligned to a single block-RAM entry, indexed by beacon ID in a 256-row dual-port memory. Three of its fields (inter-arrival time, RSSI delta, priority score) are computed on the FPGA from per-beacon state registers rather than copied from the forwarding frame, which means the feature vector is not a passive repackaging of received data but the first point at which the FPGA begins to abstract away raw packet contents in favor of classifier-ready features. The format is fixed at module boundary specification time precisely so that Module 2 (FPGA feature extraction) and Module 4 (host-side classifier training) can be developed against the same contract without either team blocking on the other—the same style of interface-first design that the framework teaches with the beacon and forwarding frames, now applied to the FPGA-to-ML boundary.

A subtle but critical fourth interface is the clock-domain boundary between the STM32 receiver's asynchronous UART output and the FPGA's 100 MHz system clock. The design crosses this boundary with a dual-flip-flop synchronizer on the `uart_rxd` input—the simplest sufficient CDC structure for a single-bit asynchronous signal—and every other signal in the ingestion pipeline is clocked by the same system clock. This is the one intentional clock-domain crossing in the Module 2 design, and it is deliberately isolated so that students can

reason about metastability in exactly one place rather than as an unbounded global concern. Without this discipline, the RF-to-FPGA interface would become a source of intermittent, difficult-to-reproduce failures that would obscure the higher-level integration problems the framework is designed to surface.

The separation of concerns between these frame and feature formats is itself a teaching point: the beacon frame is the object the transmitter firmware owns and the ML pipeline labels against, while the forwarding frame is the object the receiver firmware and the FPGA ingestion datapath own. A student who changes the beacon packet layout without coordinating the receiver's payload offset produces exactly the kind of silent cross-domain failure the framework is designed to surface.

D. Curriculum Module Sequence

Module 1: RF Hardware and Subsystem Design. Students implement beacon transmission firmware on the STM32 platform, design the packet protocol and transmission scheduling algorithm, and validate the physical receiver-to-FPGA serial interface. The primary learning objective is understanding how firmware design choices—SPI clock rate, DMA configuration, interrupt latency—affect the timing and integrity of the data stream delivered to the FPGA. The Nucleo-L476RG's four independent serial interfaces and hardware-accelerated peripherals [10] provide enough configuration space to make these tradeoffs concrete and experimentally observable.

Module 2: FPGA RTL Design and Simulation. Students implement the FPGA-side data ingestion pipeline in VHDL or Verilog: serial receiver, packet parser, CRC checker, and a feature extraction datapath. Simulation testbenches use recorded beacon data from Module 1 as stimulus. Students encounter FPGA-specific design constraints—timing closure, clock domain crossing, memory resource allocation—in the context of a real signal protocol rather than a synthetic exercise. Cloud-based FPGA toolchains have demonstrated that structured, hardware-grounded lab sequences can make this learning accessible without requiring specialized hardware beyond a web browser [9]; this module follows that model while adding the RF protocol context that a standalone digital design course omits.

Module 3: System Integration and Data Collection. Students connect the hardware subsystems built in Modules 1 and 2 and characterize end-to-end data flow: from RF beacon transmission through reception and FPGA processing to feature output. Latency, throughput, and packet error rate are measured. Synchronization mismatches and protocol errors that were invisible in per-subsystem development surface here as integration failures that must be diagnosed and resolved. Students then execute a structured data collection campaign across multiple beacon configurations, producing a labeled RF dataset. This module deliberately surfaces the distribution shift problems that arise when training data is drawn from real hardware rather than simulation [6].

Module 4: AI-Driven Classification Pipeline. Using the hardware-collected dataset from Module 3, students preprocess FPGA-extracted features, train and evaluate classifiers, and analyze classification accuracy as a function of SNR and number of concurrent beacons. Streaming inference architectures [3] and FPGA-accelerated ML pipelines [2] serve as reference designs for students considering on-device deployment. Comparative architecture studies [4] and LoRa-specific classification results [5], [6], [8] provide empirical benchmarks against which students evaluate their own model performance.

Module 5: System Validation and Benchmarking. Students stress-test the complete pipeline with multiple simultaneous beacons, measure total system response time from RF transmission to classification decision, and characterize the performance envelope of their implementation. The benchmarking exercise requires active engagement across all four domains: RF link budget analysis, FPGA timing and resource reports, serial interface throughput, and ML classification accuracy curves.

E. Pedagogical Rationale

The framework's central design principle is that the interfaces between disciplinary domains are the primary teaching objects, and this principle gives it concrete advantages over existing single-domain curricula. Prior work has demonstrated that hardware-grounded, progressive lab sequences are effective within a single discipline [9], [10]; this framework extends that validated model to four disciplines simultaneously. The key difference is that integration failures—a firmware DMA timing error that corrupts the FPGA data stream, or a beacon power change that degrades classifier accuracy—are not obstacles to be avoided but structured learning events. Each one forces the student to reason across domain boundaries in a way that no isolated course can replicate.

The framework is also designed to succeed where purely simulation-based approaches fall short. Prior work on LoRa signal classification consistently shows a gap between models trained on synthetic or benchmark data and those trained on real hardware captures [6], [8]. By staging the curriculum so that students first develop their ML pipeline on the engagement simulator's labeled synthetic data and then retrain on FPGA-collected hardware data in Module 4, the framework makes that distribution shift a deliberate teaching moment rather than a surprise. Students observe directly how channel impairments, timing offsets, and hardware non-idealities affect model performance—a lesson that cannot be delivered on a curated academic dataset.

Because the framework is intended to be adopted by instructors who may not share the author's depth in all four domains, the assessment structure is specified as concretely as the hardware. Table V shows the Module 1 assessment breakdown as a representative example: each weekly deliverable maps one-to-one onto a cross-domain interface from Section II, and the weights reflect the difficulty of the integration work rather than the volume of code produced. The trade study and hardware requirements document together account for 40% of the module grade even though they contain no executable code, because the framework treats the *justification* of a cross-

TABLE V
MODULE 1 ASSESSMENT RUBRIC (REPRESENTATIVE).

Deliverable	Weight	Due
RF module trade study report	20%	Week 1
Hardware requirements document	20%	Week 2
Beacon subsystem design	30%	Week 3
Receiver subsystem and end-to-end	30%	Week 4

domain design decision as a first-class graded artifact. This rubric structure is reproduced with analogous weights in each of the remaining four modules.

Finally, the framework is designed for reproducibility and transferability. All components—the Nexys A7-100T, the STM32 Nucleo-L476RG, the RFM95W, and the engagement simulator—are commercially available or open-source. Prior work has validated that the core technical pipelines are feasible within these hardware resource budgets [1], [3], [7], meaning an instructor at another institution can deploy the same curriculum without custom hardware or proprietary infrastructure. The framework's contribution is not a novel algorithm or hardware design but a structured, teachable model of how these four engineering disciplines connect at the system level—with each connection made concrete through implementation on real hardware.

IV. NUMERICAL RESULTS AND ANALYSIS

This section provides quantitative support for the framework by analyzing the first fully instantiated module of the curriculum (Module 1: RF Hardware and Subsystem Design). Three classes of results are presented: (1) a closed-form link-budget analysis that establishes the physical-layer operating envelope students encounter in Lab Handouts 1-D through 1-F; (2) an FPGA resource and timing budget for the Module 1 ingestion datapath on the Nexys A7-100T; and (3) performance envelope results from the engagement simulator that set the reference targets students later compare against hardware. All numerical results are derived from closed-form analysis, vendor datasheet parameters, or simulator outputs; no copied measurements are reproduced here.

A. RF Link Budget and Expected RSSI

The Module 1 baseline configuration (Lab Handout 1-D) operates the SX1276-based RFM95W at $f_c = 915$ MHz, spreading factor SF7, bandwidth $B = 125$ kHz, coding rate 4/5, and transmit power $P_{\mathrm{tx}} = +14$ dBm into a 915 MHz helical spring antenna. The free-space path loss for isotropic radiators is

$$L_{\mathrm{FSPL}}(d) = 20\log_{10}\left(\frac{4\pi d f_c}{c}\right), \tag{1}$$

which at $d = 1$ m evaluates to 31.66 dB. Assuming realistic on-breadboard spring-antenna efficiency losses of $G_{\mathrm{tx}} = G_{\mathrm{rx}} \approx -2$ dB and no additional obstruction, the expected received power is

$$P_{\mathrm{rx}} = P_{\mathrm{tx}} + G_{\mathrm{tx}} + G_{\mathrm{rx}} - L_{\mathrm{FSPL}}(1\,\mathrm{m}) \approx -21.7\ \mathrm{dBm}. \tag{2}$$

TABLE VI
MODULE 1 CLOSED-FORM LINK BUDGET AT $d = 1$ M, $f_c = 915$ MHZ, $B = 125$ KHZ, $P_{\mathrm{tx}} = +14$ DBM.

SF	Sensitivity (dBm)	P_{rx} (dBm)	Margin (dB)
SF7	−123	−21.7	101.3
SF8	−126	−21.7	104.3
SF9	−129	−21.7	107.3
SF10	−132	−21.7	110.3
SF11	−133	−21.7	111.3
SF12	−136	−21.7	114.3

Table VI tabulates the full link budget against the SX1276 LoRa sensitivity floors for representative spreading factors at $B = 125$ kHz [12]. The SF7 operating point leaves a nominal ∼101 dB of link margin at 1 m bench separation, and even the longest configured spreading factor (SF12) retains the same margin plus an additional 12.5 dB of processing gain [5], [12]. This wide margin is what makes SF7 the correct pedagogical default: students can observe clean packet reception and valid CRCs without a range environment, while still being able to reduce power or increase distance in Module 5 to recreate marginal-link behavior without changing the protocol stack.

The Module 1 acceptance criterion (≥95% packet reception, mean SNR >+5 dB, and measured mean RSSI within ±6 dB of (1)) translates this theoretical envelope into a concrete, testable outcome for every student lab group. A deviation outside the ±6 dB window forces the student to reason across disciplinary boundaries—soldered antenna joint, breadboard parasitics, or firmware PA configuration—which is the cross-domain diagnosis skill the framework exists to develop.

B. FPGA Ingestion Datapath Resource and Timing Budget

The Module 1 FPGA deliverable (Lab Handout 1-H) implements a four-block ingestion datapath on the Nexys A7-100T (Artix-7 `xc7a100tcsg324-1`): a 115200 8N1 UART receiver, a 16×8 byte FIFO, a frame-detector FSM synchronized on the `0x7E` delimiter, and a seven-segment display driver. The UART oversampling divisor at the board's 100 MHz reference clock is

$$D_{\mathrm{uart}} = \left\lfloor \frac{100 \times 10^6}{115200} \right\rfloor = 868, \tag{3}$$

which supports 16× oversampling at bit midpoints with a worst-case sampling error of under 0.06% per bit—well within the ∼2% UART tolerance, so no clock recovery is required.

Table VII summarizes the projected resource envelope against the `xc7a100t` device totals [11]. The datapath fits in under 1% of LUT and FF resources and uses none of the 240 DSP slices or the ≈4860 Kb of block RAM available on the device. This intentional under-utilization is a pedagogical feature: Modules 2 and 4 reuse the same platform to add a CRC-16/CCITT-FALSE checker, feature extraction, and on-device inference, and the Module 1 skeleton must leave headroom for those blocks rather than consume the device.

Timing closure for this datapath is effectively unconstrained: the longest combinational path (frame-detector next-state logic) crosses fewer than ten LUT levels, and the target is

TABLE VII
PROJECTED MODULE 1 INGESTION DATAPATH RESOURCE BUDGET ON `xc7a100tcsg324-1`.

Block	LUTs	FFs	BRAM (Kb)
`uart_rx`	∼85	∼60	0
`byte_fifo` (16×8)	∼45	∼35	0
`frame_detector`	∼120	∼95	0
`seg7_driver`	∼55	∼40	0
Module 1 total	∼305	∼230	0
Device available	63,400	126,800	4,860
Utilization	<0.5%	<0.2%	0%

TABLE VIII
ENGAGEMENT SIMULATOR STEADY-STATE PERFORMANCE ENVELOPE (DEFAULT SCENARIO, 60-TICK ROLLING WINDOW).

Metric	Theoretical	Expected range
Communication success rate	87.5%	80–92%
Jamming effectiveness	12.5%	8–18%
Counter-jamming success	40.0%	32–48%

a single 100 MHz clock domain with no CDC. Worst negative slack (WNS) is therefore positive by a wide margin. This result is important pedagogically because it means that when students *do* encounter negative slack in Module 2 after adding CRC and feature-extraction logic, the regression is attributable to their own additions rather than to the skeleton—an essential property for a learning environment.

C. Engagement Simulator Performance Envelope

The engagement simulator (Section III) provides a controlled, labeled performance envelope against which students calibrate expectations before physical hardware is available. With the default parameters—eight FHSS channels at 225 MHz, 500 ms tick, two cooperative target stations sharing a seed-synchronized hop sequence, a wideband DRFM-style jammer, and a 40% counter-jamming success probability—the steady-state outcome of each tick is fully determined by three independent events: channel alignment between targets (probability $1/8$ per tick absent hopping collisions), successful jamming given channel overlap, and successful counter-jamming given jamming detection.

Under this model the expected steady-state communication success rate, jamming effectiveness, and counter-jamming rate take closed-form values that the simulator reproduces over its 60-tick rolling window. Table VIII reports the theoretical values alongside the operating envelope students are expected to observe. These numbers establish the Module 4 classifier baseline: a supervised model trained on simulator log data should converge on counter-jamming classification accuracy approaching the 40% success-rate ceiling imposed by the stochastic model, not higher. Students who report significantly higher accuracy are almost certainly overfitting to the log's structural artifacts—a lesson that is only teachable because the simulator's ground truth is known in closed form.

TABLE IX
PROJECTED RESOURCE BUDGET FOR THE COMPLETE MODULE 2 PIPELINE ON `xc7a100tcsg324-1`.

Block	LUTs	FFs	BRAM
Dual-FF synchronizer	2	2	0
UART receiver	80	50	0
Receive FIFO (64×8)	20	15	0
Packet parser FSM	200	300	0
CRC-8 engine	30	10	0
CRC-16 engine	50	20	0
Feature extraction	150	200	0
Per-beacon state RAM	0	0	1
Feature BRAM	0	0	4
Host UART export	60	40	0
Module 2 total	**600**	**640**	**5**
Device available	63,400	126,800	270
Utilization	<1%	<1%	∼2%

D. Module 2 Pipeline Resource, Timing, and Verification Coverage

The Module 1 ingestion skeleton analyzed earlier in this section is deliberately minimal; the full pipeline that the framework targets—including dual-flip-flop synchronizer, UART receiver, byte FIFO, packet parser FSM, CRC-8 transport check, CRC-16 payload check, feature extraction datapath, per-beacon state RAM, feature BRAM, and host UART export—is specified in Module 2. Quantifying this full pipeline is important because it is the state in which the platform enters Module 4 (machine learning), and any resource or timing headroom claim about the framework must be made at that scale, not at the Module 1 skeleton scale.

Table IX shows the block-level resource budget for the complete Module 2 pipeline targeting `xc7a100tcsg324-1`. Totals are approximately 600 LUTs, 640 flip-flops, and five 18 Kb BRAM tiles. The two largest BRAM consumers are the 256-row per-beacon state memory and the 256-row × 256-bit feature storage, both indexed directly by beacon ID so that no hashing or associative lookup is required. Against device totals of 63,400 LUTs, 126,800 flip-flops, and 270 BRAM tiles, aggregate utilization remains under 1% of logic and approximately 2% of block RAM—leaving essentially the entire device available for the Module 4 inference engine.

Timing closure at the target 100 MHz system clock is achieved with worst negative slack greater than 3 ns in the reference implementation, and dynamic power dissipation is projected below 200 mW. The critical path falls on the CRC-16 XOR tree; if it does not close on a given student's implementation, the standard remediation is a single pipeline register splitting the XOR network across two cycles, which reclaims several nanoseconds of slack at no resource cost. The only intentional clock-domain crossing in the design is the dual-flip-flop synchronizer on `uart_rxd`, isolating metastability to a single well-defined structure.

Verification coverage is specified as an eight-scenario regression suite that every Module 2 submission must pass. Table X summarizes the scenarios and their pass criteria. The suite deliberately pairs positive cases (normal operation,

TABLE X
MODULE 2 EIGHT-SCENARIO VERIFICATION SUITE.

#	Scenario
1	Normal operation (2 beacons, 20 frames each)
2	CRC-8 transport failure (1 of 20 corrupted)
3	CRC-16 beacon payload failure
4	Garbage bytes before valid `0x7E` delimiter
5	Back-to-back frames (zero gap)
6	Maximum burst (FIFO pressure test)
7	Long-duration single beacon (100 frames)
8	Beacon ID collision / overwrite

long-duration stability, multi-beacon interleaving) with fault-injection cases (CRC-8 transport corruption, CRC-16 payload corruption, leading garbage bytes, maximum-burst FIFO pressure, and beacon ID collision) so that a student cannot pass by exercising only the happy path. The existence of a fixed regression suite also gives instructors at other institutions an objective basis for evaluating reproducibility: two independent implementations of the framework can be compared by running both against the same stimulus files and comparing feature-vector outputs byte-for-byte.

E. Module 1 Acceptance Criteria as Quantitative Outcomes

The three analyses above are tied together by a set of pass/fail criteria that constitute the Module 1 quantitative deliverable. A student cohort completes Module 1 successfully when, across all lab groups: (i) every assembled RFM95W breakout draws between 0.5 and 5 mA of idle current at 3.3 V and reads `RegVersion = 0x12`; (ii) the two-node link test achieves $\geq$95% reception with zero CRC errors across 100 consecutive packets and mean RSSI within $\pm$6 dB of the Table VI prediction; (iii) the dual-beacon concurrent run (SF7 at +14 dBm, SF10 at +10 dBm) demonstrates quasi-orthogonal demodulation with independently contiguous sequence counters on both streams; and (iv) the Nexys A7 end-to-end display updates within 2 seconds of a beacon being added or removed, with zero Vivado implementation timing violations. Each of these criteria corresponds to one of the four cross-domain interfaces defined in Section II, so a failure at any criterion isolates the affected interface for remediation without ambiguity.

V. CONCLUSION

This paper presented an educational framework that treats the interfaces between FPGA digital design, RF communications, embedded firmware, and machine learning as first-class teaching objects. The framework is instantiated by a benchtop platform built from commercially available components—STM32 Nucleo-L476RG microcontrollers, RFM95W LoRa transceivers, and a Digilent Nexys A7-100T FPGA—and a five-module progressive curriculum that sequences students through hardware bring-up, RTL design, system integration, AI-driven classification, and full-system benchmarking. A Python engagement simulator with a physics-grounded FSPL link model and known ground-truth labels bridges the gap between curriculum introduction and hardware availability, and later serves as the controlled baseline against which real-hardware machine learning results are compared. Quantitative analysis of the first fully instantiated module shows that the chosen hardware leaves more than 100 dB of link margin at bench distances while still exposing every LoRa physical-layer parameter as a tunable student-facing variable, that the Module 1 FPGA ingestion datapath consumes under 0.5% of the Artix-7 `xc7a100t` LUT budget and leaves the entire DSP and block-RAM resource pool available for the classification pipeline added in Modules 2 and 4, and that the engagement simulator's closed-form performance envelope provides a rigorous overfitting check for student-trained classifiers. The Module 1 skeleton deliberately consumes only a small fraction of the Artix-7 device so that the CRC-16 checker, feature extraction datapath, and streaming inference blocks added in Modules 2 and 4 can be introduced as student additions rather than replacements—preserving the framework's integration-first pedagogy through the entire module sequence. Taken together, these results demonstrate that the framework is not only pedagogically coherent but also technically feasible within the resource and complexity budgets of a graduate-level capstone course, and that an instructor adopting the framework at another institution can reproduce both the hardware platform and the quantitative acceptance criteria directly from the specification given here.

REFERENCES

[1] H. Komkov, L. Pocher, A. Restelli, B. Hunt, and D. Lathrop, "RF signal classification using Boolean reservoir computing on an FPGA," in *Proc. 2021 Int. Joint Conf. Neural Networks (IJCNN)*, Jul. 2021, doi: 10.1109/IJCNN52387.2021.9533342.

[2] J. F. Lin, T. Morehouse, C. Montes, E. Caushi, A. Dudko, E. Savage, and R. Zhou, "A study of angle of arrival estimation of a RF signal with FPGA acceleration," in *Proc. 2024 Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, 2024, pp. 1625–1628, doi: 10.1109/IWCMC61514.2024.10592551.

[3] A. Maclellan, L. H. Crockett, and R. W. Stewart, "Streaming convolutional neural network FPGA architecture for RFSoC data converters," in *Proc. 2023 21st IEEE Interreg. NEWCAS Conf. (NEWCAS)*, Edinburgh, UK, Jun. 2023, doi: 10.1109/NEWCAS57931.2023.10198198.

[4] K. A. Alnajjar, S. Ghunaim, and S. Ansari, "Automatic modulation classification in deep learning," in *Proc. 2022 5th Int. Conf. Commun., Signal Process., Appl. (ICCSPA)*, Cairo, Egypt, Dec. 2022, doi: 10.1109/ICCSPA55860.2022.10019122.

[5] P. M. Mutescu, A. Lavric, A. I. Petrariu, and V. Popa, "Deep learning enhanced spectrum sensing for LoRa spreading factor detection," in *Proc. 2023 13th Int. Symp. Adv. Topics Elect. Eng. (ATEE)*, Bucharest, Romania, Mar. 2023, doi: 10.1109/ATEE58038.2023.10108224.

[6] Y. E. Sagduyu and T. Erpek, "Adversarial attacks on LoRa device identification and rogue signal detection with deep learning," in *Proc. MILCOM 2023 — 2023 IEEE Military Commun. Conf.*, Boston, MA, Oct.–Nov. 2023, pp. 385–390, doi: 10.1109/MILCOM58377.2023.10356224.

[7] T.-P. Dang, T.-K. Tran, T.-T. Bui, and H.-T. Huynh, "LoRa gateway based on SoC FPGA platforms," in *Proc. 2021 Int. Symp. Elect. Electron. Eng. (ISEE)*, 2021, pp. 48–52, doi: 10.1109/ISEE51682.2021.9418711.

[8] K. Dakic, B. Al Homssi, A. Al-Hourani, and M. Lech, "LoRa signal demodulation using deep learning, a time-domain approach," in *Proc. 2021 IEEE 93rd Veh. Technol. Conf. (VTC2021-Spring)*, Apr. 2021, doi: 10.1109/VTC2021-Spring51267.2021.9448711.

[9] W. Smith, Z. Driskill, J. Goeders, and M. Wirthlin, "Digital design education using an open-source, cloud-based FPGA toolchain," in *Proc. 2024 Intermountain Eng., Technol. Comput. Conf. (IETC)*, May 2024, doi: 10.1109/IETC61393.2024.10564285.

[10] N. O. Strelkov, V. V. Krutskikh, and E. V. Shalimova, “Programming STM32 Nucleo platform for IoT education using STM32duino and Mbed OS,” in *Proc. 2022 VI Int. Conf. Inf. Technol. Eng. Educ. (Inforino)*, Apr. 2022, doi: 10.1109/Inforino53888.2022.9782920.

[11] Digilent Inc., “Nexys A7 FPGA Board Reference Manual,” Digilent Inc., Pullman, WA, 2023. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[12] Adafruit Industries, “Adafruit RFM69HCW and RFM9X LoRa Packet Radio Breakouts,” Adafruit Industries, New York, NY, 2023. [Online]. Available: https://learn.adafruit.com/adafruit-rfm69hcw-and-rfm96-rfm95-rfm98-lora-packet-padio-breakouts/downloads

[13] STMicroelectronics, “STM32 Nucleo-64 Boards (MB1136) User Manual,” UM1724, STMicroelectronics, Geneva, Switzerland, 2023. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-l476rg.html